

Week 42, Tuesday Session

Flipped classroom: your questions, then one case – the backward pass of a neural network,
step by step

Bring your laptop; open `week42tuesday.ipynb` and `exercisesweek42.ipynb`

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

How today works

The format (as every Tuesday)

Mondays are lectures; Tuesdays belong to your questions and to concrete cases. Today there is *one* case, and it is the skeleton of this week's exercises.

1. **First ~20 min:** your open questions from Monday's lecture – the four equations and the output heads once more, the trial solution and the residual as a cost, derivatives with respect to the input, the diffusion equation, soft against hard constraints, the inverse problem – discussed in plenum.
2. **Rest of the double session:** Case 1, worked in groups with `week42tuesday.ipynb`: the gradient of a network by hand in six steps – the shapes and one layer, two layers, any number of layers, the batched version, why the activation functions, and (optional) a classifier with the softmax shortcut and SGD with momentum – every step checked against `jax.grad`. Then the insight questions in plenum.
3. **Last 5 min:** wrap-up, the exercises, and what Monday builds on.

Exercises week 42

The same six steps on the networks of `exerciseweek42.ipynb`, then training on the iris data. Deadline **Friday October 16 at midnight**; declare your use of LLMs (template at the end of the notebook).

Picking up from Monday

Monday in one sentence each

The forward pass stores every Z^l and A^l ; the backward pass starts with $\Delta^L = A^L - Y$ for the three natural pairings, pulls the error back through W^T and modulates by f' , and the gradients are $(A^{l-1})^T \Delta^l$ and the column sums. The hidden activation must be non-polynomial and enters only through f' – and through f'' once the output is differentiated with respect to the input, which is why the differential equations used tanh. A trial solution carries the conditions exactly, the mean squared residual is the cost, and the same network solved decay, logistic growth, Poisson and the diffusion equation; soft constraints lose to hard ones when both exist, and win when they are the only option.

Questions we expect

Why f'' , and why is ReLU's exactly zero? Why is the error at $t = 0$ *exactly* zero for the trial solution? Why did the 20-unit decay network get stuck at 31.9? What are the λ in the PINN cost, and why did $\lambda = 100$ make things worse? How can forty points replace the boundary conditions? Ask.

Today's single question

Monday's backpropagation was eight lines. Today you write those lines yourself, from the chain rule, with the shape of every object written down before it is computed, and with `jax.grad` as the referee at every step – because the exercises and Project 2 ask for exactly that, and because a backward pass you have not written is a backward pass you cannot debug.

Case 1: the backward pass, six steps on examples of our own

Goal: leave with your own backpropagation, in both weight conventions, agreeing with `jax.grad` to machine precision, and with the shapes of every gradient in your head (notebook, Case 1; seed 2026). The exercises are the same steps on their own networks.

1. **Shapes first, then one layer by hand:** W of shape $(4, 3)$, x of shape $(3,)$, sigmoid, mean squared error. Write the shapes of $\partial C / \partial W$ and $\partial C / \partial b$ *before* running; then $\delta = (\partial C / \partial a) \circ \sigma'(z)$, the outer product, the check.
2. **Two layers:** a 3–4–2 network; the last layer is step 1 with a_1 in place of x ; the first needs $\partial C / \partial a_1 = W_2^T \delta_2$ – the pull-back, Eq. (8.64).
3. **Any number of layers:** `feed_forward_saver` keeps every layer input and every z ; one loop from the last layer to the first, Eqs. (8.65)–(8.68), with `[sigmoid, ReLU, sigmoid]` on a 3–5–4–2 network.
4. **Batched inputs:** samples in rows, W of shape (inputs, outputs); the three lines that change. Check that the batched gradient is the *mean* of the single-sample gradients.
5. **Why the activation functions:** fit the Runge function with a 1–20–1 network three times – identity, sigmoid, ReLU hidden units – with the same backward pass.
6. **Optional:** the digits data, `[sigmoid, softmax]`, the shortcut $\delta^L = a^L - y$ of Eq. (8.69), SGD with momentum.

Checkpoints (plenary after ~ 40 min)

Step 1: $(4, 3)$ and $(4,)$, agreement 10^{-17} . Step 2: $|\delta_1|/|\delta_2| = 0.20$. Step 3: the ReLU node with $z < 0$ has $\delta = 0$ exactly. Step 4: batched = mean of ten single-sample gradients. Step 5: identity $0.1024 = \text{var}(y)$, sigmoid 0.0164 , ReLU 0.0144 . Step 6: $9.7\% \rightarrow 96.4\%$ test accuracy in 50 epochs.

Case 1: the code you should end up with

```
# step 1: one layer, single-vector convention, W of shape (outputs,
inputs)
z = W @ x + b; a = sigmoid(z) # forward, keep z and a
dC_da = mse_der(a, target) # 2 (a - y)/n_out
dC_dz = dC_da * sigmoid_der(z) # the error delta
dC_dW = np.outer(dC_dz, x) # (4,) x (3,) -> (4, 3)
dC_db = dC_dz # (4,)
W_g, b_g = jax.grad(cost_one_layer, argnums=(0, 1))(W, b, x, target)

# step 2: the first of two layers
dC_da1 = W2.T @ dC_dz2 # (4, 2) @ (2,) -> (4,) Eq. (8.64)
dC_dz1 = dC_da1 * sigmoid_der(z1)
dC_dW1 = np.outer(dC_dz1, x); dC_db1 = dC_dz1
```

```
# step 3: any number of layers
def backpropagation(input, layers, activation_funcs, target,
                    activation_ders, cost_der=mse_der):
    layer_inputs, zs, predict = feed_forward_saver(input, layers,
                                                    activation_funcs)
    layer_grads = [()] for layer in layers]
    for i in reversed(range(len(layers))):
        layer_input, z, activation_der = layer_inputs[i], zs[i],
            activation_ders[i]
        if i == len(layers) - 1:
            dC_da = cost_der(predict, target) # (8.65): from the cost
        else:
            (W, b) = layers[i + 1]
            dC_da = W.T @ dC_dz # (8.66): from the layer above
            dC_dz = dC_da * activation_der(z) # the error of layer i
            dC_dW = np.outer(dC_dz, layer_input) # (8.68)
            dC_db = dC_dz # (8.67)
            layer_grads[i] = (dC_dW, dC_db)
    return layer_grads
```

```
# step 4: batched -- samples in rows, W of shape (inputs, outputs)
# the three lines that change:
dC_da = dC_dz @ W.T # (n, M_{l+1}) @ (M_{l+1}, M_l)
dC_dW = layer_input.T @ dC_dz # summed over the samples
dC_db = np.sum(dC_dz, axis=0)

# step 6: softmax + cross entropy, the shortcut of Eq. (8.69)
if i == len(layers) - 1:
    dC_dz = (predict - targets)/n # no Jacobian, no 1/a
```

Three things to get right

Save the forward pass: (8.66) needs $f'(z_l)$ and (8.68) needs the layer input, at the parameters the forward pass used. **The transpose:** $W^T \delta$ one vector at a time, ΔW^T for a batch – the sum over the *receiving* nodes either way. **The normalisation:** mse takes the mean, so mse_der carries $2/n$; without it the check fails, and a gradient that is right up to a constant is a bug a learning rate hides.

The referee

`jax.grad(cost, argnums=0)(layers, ...)` differentiates with respect to a *list of tuples of arrays* and returns the same structure: a list of (dW, db) with the shapes of the layers. `np.allclose` on every pair, at every step, before you move on.

Case 1: what you should find

Steps 1–4: the shapes and the numbers

Step 1: $\partial C / \partial W$ of shape (4, 3), $\partial C / \partial b$ of shape (4,) – the shapes of W and b , one number per parameter; $\delta = (0.012, 0.026, -0.039, -0.046)$; largest difference to `jax.grad` 1.4×10^{-17} . Step 2: four gradients, four shapes, all agree; $|\delta_1|/|\delta_2| = 0.20$ – the error shrinks through W_2^T and $\sigma' \leq \frac{1}{4}$, and ten such layers would shrink it by 4^{-10} . Step 3, 3–5–4–2 with [sigmoid, ReLU, sigmoid]: the ReLU layer has $z = (0.23, -1.15, -0.87, 0.80)$ and $\delta = (-0.0093, 0, 0, -0.0002)$ – the two nodes with $z < 0$ deliver exactly nothing to their weights and to the layer below. Step 4: shapes (3, 5), (5, 4), (4, 2), agreement with `jax.grad` of the batched cost; the batched $\partial C / \partial W_1$ is the mean of the ten single-sample gradients.

Step 5: remove the activation functions

1–20–1, 10 000 steps of plain gradient descent on the Runge data:

hidden activation	γ	training MSE
identity	0.05	$0.1024 = \text{var}(y)$
sigmoid	0.2	0.0164
ReLU	0.1	0.0144

The identity network collapses to $-0.02x + 0.35$: 41 parameters, a straight line at the mean of y . Twenty smooth steps or twenty kinks bend towards the noise floor 0.01. The same backward pass in all three cases.

Step 6: the digits

64–32–10, [sigmoid, softmax], the shortcut checked against `jax.grad` of the cross entropy (agrees), SGD with momentum $\gamma = 0.1$, $\beta = 0.9$, $M = 32$: test accuracy 9.7% $\rightarrow$ 65.6% after one epoch, 93.6% after five, 96.4% after fifty – against 99.9% on the training set: the gap is the variance term.

Case 1: do you have the insight?

1. **Shapes.** W has shape $(10, 4)$ and b shape $(10,)$ in the single-vector convention. What are the shapes of $\partial C / \partial W$, $\partial C / \partial b$ and δ ? With n samples in rows, which shapes change, and which products carry the sum over the samples?
2. **What is shared.** Which quantity do $\partial C / \partial W$ and $\partial C / \partial b$ of the same layer have in common, and why must the forward pass save every z and every layer input? What goes wrong if you recompute the forward pass after updating the last layer?
3. **The pull-back.** Why does the error travel backwards with W^T and not with W ? What does a ReLU node with $z < 0$ deliver to the layer below, and a sigmoid node with $|z| = 10$?
4. **Why the activation functions.** The identity network fitted a line, the sigmoid and ReLU networks the Runge function. Which property of linear maps is responsible, what does the universal approximation theorem require of the activation, in what sense is a ReLU network a sum of kinks – and does the backward pass “know”?

Case 1: do you have the insight?

1. **Shapes.** W has shape $(10, 4)$ and b shape $(10,)$ in the single-vector convention. What are the shapes of $\partial C / \partial W$, $\partial C / \partial b$ and δ ? With n samples in rows, which shapes change, and which products carry the sum over the samples?
2. **What is shared.** Which quantity do $\partial C / \partial W$ and $\partial C / \partial b$ of the same layer have in common, and why must the forward pass save every z and every layer input? What goes wrong if you recompute the forward pass after updating the last layer?
3. **The pull-back.** Why does the error travel backwards with W^T and not with W ? What does a ReLU node with $z < 0$ deliver to the layer below, and a sigmoid node with $|z| = 10$?
4. **Why the activation functions.** The identity network fitted a line, the sigmoid and ReLU networks the Runge function. Which property of linear maps is responsible, what does the universal approximation theorem require of the activation, in what sense is a ReLU network a sum of kinks – and does the backward pass “know”?

What we hope you say

(1) The gradient of a scalar with respect to an array has the shape of that array: $(10, 4)$, $(10,)$, and δ the shape of z ; batched, δ becomes $(n, 10)$, the gradients keep their shapes, and `layer_input.T @ dC_dz` and `sum(dC_dz, axis=0)` sum over the samples. (2) The error $\delta = \partial C / \partial z$: the bias gradient is δ , the weight gradient is δ times the layer input; (8.66) needs $f'(z_l)$ and (8.68) needs a_{l-1} at the parameters the forward pass used – recomputing after an update mixes two networks and the gradient is of neither. (3) $z_{l+1} = W a_l + b$ sends node j to every node k with weight W_{kj} ; collecting the contributions is $\sum_k W_{kj} \delta_k = (W^T \delta)_j$, the sum over the receiving index. A dead ReLU node delivers exactly zero, a saturated sigmoid node $\sigma'(10) \approx 5 \times 10^{-5}$ of what it received: the two faces of the vanishing gradient. (4) The composition of linear maps is linear – $W_1 W_2$ is a matrix – so depth without non-linearity adds parameters but no functions; the theorem needs a non-polynomial activation; each ReLU unit is a kink at $-b_i / w_i$ and the output layer adds kinks with weights (Lemma 8.1). The backward pass is the chain rule and works for any differentiable model; the non-linearity lives in the model, the optimiser only follows the gradient.

Wrap-up, the exercises, and Monday

What you should own now

A backward pass you wrote yourself, in both weight conventions, with every shape in your head and `jax.grad` as the referee; the three lines that change when the samples go into rows, the one line that changes for softmax + cross entropy; and the reason a network without activation functions is a linear model however deep it is – while the backward pass does not care.

Exercises week 42 – deadline Friday October 16 at midnight

`exercisesweek42.ipynb`: the same six steps on the networks of the exercise set, with the `jax.grad` checks built into the notebook, then training on the iris data with your own backpropagation and SGD with momentum, and the optional network class. Hand in one notebook per group in Canvas; list your collaborators; finish the self-check table; declare your use of LLMs.

Project 2 (deadline November 6) and next Monday

Project 2 is built on exactly this code: the batched backward pass of step 4 with the three heads of Monday's first half hour, on the Runge data and on the full MNIST set. Next Monday, week 43: convolutional neural networks (Chapter 10) – keeping the spatial structure we threw away when we flattened 28×28 into 784.